

Concurrent Real-Time Competition Leaderboard System

CS F213 — Object Oriented Programming · Project Part B Report

Group	28
Members	Aayush Rastogi (2023B2A30888H) · Shaan Sharma (2023B1AD0949H)
Problem track	3 — Problem Solving with Algorithms and Data Structures
Code repository	https://github.com/thegreatflash/oop-project-group28

1. Problem definition

During multi-stage festival competitions (dance, debate, quiz), judges enter scores concurrently from tablets. The existing platform updates its leaderboard by recalculating every rank from scratch on each incoming score — an $O(n \log n)$ full re-sort — which causes visible lag exactly when load peaks (finals, several judges submitting at once). Our module replaces this with a thread-safe ranking engine whose cost per update is $O(\log n)$, plus a web interface for judges, participants, and administrators.

1.1 Inputs

- `data/participants.csv` — `competitionId, competitionName, participantId, participantName` (produced by the event-management module).
- `data/judges.csv` — `judgeId, judgeName`.
- Score submissions (`competitionId, participantId, judgeName, value`) arriving concurrently from the Judge web page or the built-in load simulator.

1.2 Outputs

- Live, ordered standings per competition (full board and Top-K), auto-refreshing in the browser every 3 seconds.
- `data/<competition>_leaderboard.csv` — exported standings for the festival backend.
- `data/saved_scores.csv` — every judge's individual score, written on shutdown and restored on start (crash/restart recovery).

1.3 Constraints and scoring rules

- Scores are whole numbers in $[0, 100]$; anything else is rejected with a custom checked exception and never reaches the ranking engine.
- Each judge holds exactly **one** score per participant — resubmitting replaces the earlier value. A participant's displayed score is the **average** of their judges' scores with the decimal dropped, so it can never exceed 100.
- Ranking state must stay correct under concurrent writes with no global lock; readers must never block writers.
- Role-based access: participants view, judges score, administrators manage.

1.4 Success metrics

- Correct, race-free rankings under concurrent submission — proven by JUnit stress tests (hundreds of parallel submissions, zero lost updates).
- Score update cost $O(\log n)$ reposition instead of $O(n \log n)$ re-sort; Top-K retrieval $O(k)$.
- Leaderboard visible to viewers within one 3-second refresh cycle of a submission.
- State survives an application restart via file persistence.

1.5 Scope

As scoped for 2–3 weeks, the simulation runs 3 competitions $\times$ 20 participants with 3 judges. The engine itself is size-independent — the data structures and concurrency design are the deliverable, and the simulator can drive continuous load to demonstrate behaviour under concurrency.

2. System architecture

The application is a single Spring Boot process organised in four packages with a strict direction of dependency (web $\rightarrow$ core/io $\rightarrow$ model). The engine (`model`, `core`, `io`) is plain Java with no framework imports, so it is testable and reusable outside the web layer.

Package	Classes	Responsibility
<code>model</code>	Person (abstract), Participant, Judge, Competition, Score (record)	Domain objects; inheritance hierarchy and immutable score value.
<code>core</code>	Leaderboard<T extends Person>, LeaderboardManager, ScoreProcessor, ScoreSimulator, Ranking<T> (record), InvalidScoreException	Ranking engine, concurrency, validation, observer notifications, load simulation.
<code>io</code>	CSVManager	All file exchange: load participants/judges, export standings, save/restore judge scores.
<code>web</code>	LeaderboardService, PageController, SecurityConfig	Spring layer: pages (Thymeleaf), JSON refresh endpoint, role-based access control.

2.1 Score lifecycle

1. A judge submits the form (or the simulator generates a score).
2. `ScoreProcessor.submit()` validates on the caller's thread — out-of-range or non-integer values throw `InvalidScoreException` immediately, so the judge sees the error at once.
3. Valid scores are executed on a fixed 3-thread `ExecutorService` (producer–consumer: the pool's queue is the buffer).
4. `Leaderboard.addScore()` stores the judge's value and repositions that one participant in the ordered index — $O(\log n)$.
5. `LeaderboardManager` fires a `PropertyChangeEvent` (observer pattern); the browser picks up the new standings on its next poll.

3. Core design: incremental ranking and concurrency

3.1 Data structures

Two structures cooperate inside `Leaderboard<T extends Person>`:

- `ConcurrentHashMap<T, Map<String, Double>>` — participant $\rightarrow$ (judge $\rightarrow$ latest score). Holds the authoritative scoring data.
- `ConcurrentSkipListMap<RankKey, T>` — the live ranking index, keyed by `(score desc, name, id)`. The id makes keys unique even for equal names and scores.

A score update removes the participant's old key and inserts the new one — two $O(\log n)$ operations. Reading the standings is a walk of the already-ordered map (no sorting on the read path), and `getTopK(k)` returns the first k entries in $O(k)$.

Operation	Naive design	This design
Score update	$O(n \log n)$ full re-sort	$O(\log n)$ reposition
Ranking refresh	Full recomputation	Walk of ordered index
Top-K retrieval	Sort then slice	$O(k)$ prefix read
Concurrent updates	Global lock, writers serialise	Per-participant locking, parallel across participants

3.2 Thread-safety strategy

Every update runs inside `ConcurrentHashMap.compute()`, which locks only that participant's entry. Two judges scoring the *same* participant are applied strictly one after the other — so the skip-list reposition can never leave a stale key — while judges scoring *different* participants proceed fully in parallel. There is no `synchronized` block and no global lock anywhere; readers (the standings walk) never block writers. Spring Boot's request threads plus the 3-worker pool mean submissions genuinely arrive concurrently, which is exactly what the stress tests exercise.

4. Security design

Spring Security provides form login with three roles backed by an in-memory user store (passwords BCrypt-hashed):

Accounts	Role	Access
participant1–3 / 1234	PARTICIPANT	Home, live leaderboard
judge1–3 / 1234	JUDGE	+ Judge page (submit scores)
admin / 1234	ADMIN	+ Admin page (reset, export, simulator)

URL rules live in `SecurityConfig (/judge/** requires JUDGE or ADMIN, /admin/** requires ADMIN)`. Unauthorised access returns 403.

5. Advanced Java features used

Each feature below corresponds to a unit of the CS F213 syllabus — the collections framework, multithreading and concurrency, generics, exception handling, file I/O, lambda expressions, and design patterns, as covered in the course lectures and handout.

Feature	Where and why
Collections framework & concurrent collections	<code>ConcurrentHashMap</code> (scores, competitions), <code>ConcurrentSkipListMap</code> (ordered ranking index) — lock-free reads, fine-grained locked writes; plus <code>List/Map</code> throughout.

Multithreading	Fixed-pool <code>ExecutorService</code> for score processing; <code>ScheduledExecutorService</code> driving the simulator; <code>CountDownLatch</code> in the concurrency tests; atomic <code>compute()</code> updates.
Generics	<code>Leaderboard<T extends Person></code> and <code>Ranking<T></code> — bounded type parameter lets the same engine rank any person type without casting.
Exception handling	Custom checked <code>InvalidScoreException</code> ; validation before queueing so a bad input can never corrupt shared state; graceful handling of malformed CSV rows.
File I/O	<code>java.nio.file</code> in <code>CSVManager</code> : load, export, save/restore across restarts.
Records & immutability	<code>Score</code> , <code>Ranking</code> , <code>RankKey</code> — immutable data passed safely between threads.
Lambdas & method references	Executor tasks, <code>compute()</code> remapping functions, comparators, stream mapping in the JSON endpoint.
Design patterns	Observer (<code>PropertyChangeSupport</code> in <code>LeaderboardManager</code>), Producer-Consumer (submission pipeline), MVC (Spring controller/service/templates), dependency injection.
Framework: Spring Boot + Spring Security	Embedded web server, Thymeleaf templating, role-based access control, BCrypt password hashing.

6. Integration with the festival platform

Integration is file-based (one of the modes permitted by the specification): the module consumes CSVs that the event-management/registration modules would produce, and emits CSVs the dashboard and analytics modules can ingest. The JSON endpoint (`/api/leaderboard?competition=<id>[&top=k]`) additionally provides a REST-style read interface that a mobile app could poll. Swapping CSV exchange for REST or JDBC would only touch `CSVManager` and the service layer — the engine is unaware of the transport.

7. Testing

`mvn test` runs **20 JUnit 5 tests**, all passing:

Test class	Coverage
LeaderboardTest (8)	Average-of-judges scoring, same-judge replacement, rank ordering and overtaking, Top-K, ties by name, reset keeping participants, bounds never exceeding 100.
ScoreProcessorTest (5)	Rejection of negative, >100 and non-integer scores; boundary values 0 and 100 accepted; rejected scores leave the board untouched.
ConcurrentSubmissionTest (3)	Hundreds of submissions from up to 8 threads: no lost updates through the processor pipeline; skip-list index stays consistent (each participant exactly once, correct order) under concurrent repositioning.
CSVManagerTest (4)	Loading with comment/blank/malformed rows, judge loading, export format and rank order, save→restart→restore round trip.

8. Code repository and how to run

Full source, unit tests, sample data, UML and this report: <https://github.com/thegreatflash/oop-project-group28>. Requires JDK 21+; Maven is not needed (the bundled wrapper downloads it).

```
git clone https://github.com/thegreatflash/oop-project-group28
cd oop-project-group28
./mvnw spring-boot:run      # then open http://localhost:8080
./mvnw test                 # run the 20 unit tests
```

Run from the project root so the `data/` folder is found, and sign in with the accounts from section 4 (e.g. `admin / 1234`). To run with no local setup: repository page → **Code** → **Codespaces** → **Create codespace on main**, then the same commands in its terminal (open forwarded port 8080). To demonstrate the simulated integration environment, start the simulator from the Admin page and watch the leaderboard reorder; exports land in `data/` as CSV. Full step-by-step detail is in `HOW_TO_RUN.md` in the repository.

9. Technology stack

- **Java 21** (records, lambdas, concurrent collections) · **Maven** with wrapper
- **Spring Boot 3.3** (web, Thymeleaf) · **Spring Security** (roles, BCrypt)
- **JUnit 5** for tests · **PlantUML** for the class diagram
- **GitHub + Codespaces** for hosting, collaboration and cloud runs
- AI-assisted development tooling was used during implementation, as permitted by the project guidelines encouraging modern technology use.

10. Innovations beyond the base requirements

- Incremental $O(\log n)$ ranking with $O(k)$ Top-K, demonstrated live in the UI.
- Role-based security with three user classes.
- Built-in concurrent load simulator for demonstrations and stress testing.
- Restart-safe persistence of every judge's individual score.
- Judge-score semantics (replace + average) preventing score inflation.

11. Member contributions

Member	Modules / responsibility
Aayush Rastogi (2023B2A30888H)	Domain model and core ranking engine: the <code>Leaderboard</code> skip-list design and scoring rules, <code>ScoreProcessor</code> thread pool and validation, observer wiring in <code>LeaderboardManager</code> ; Home, Leaderboard and Judge pages with the auto-refresh script; concurrency and ranking unit tests.
Shaan Sharma (2023B1AD0949H)	Security module (<code>SecurityConfig</code> , roles, login page) and UI theme; Admin module with reset/export and the load simulator; <code>CSVManager</code> file integration and restart persistence; CSV and score-validation unit tests; report and demo preparation.

Appendix A — UML class diagram

Competition Leaderboard - Class Diagram

